CS & IT ENGINEERING

Algorithms

Dynamic Programming

Lecture No.- 06

By- Aditya sir

Recap of Previous Lecture

- Topic
- Topic

LCS

MCM

Topics to be Covered

- Topic
- Topic
- Topic

mcm

SOS

About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.

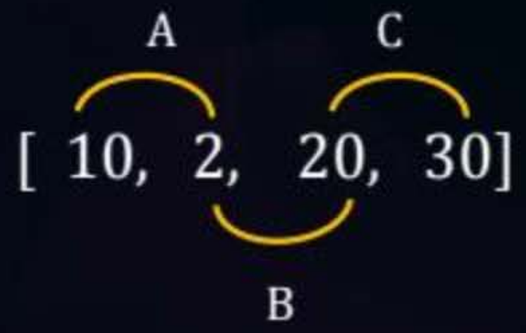
Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW

Topic : Dynamic Programming

DP based approach:-

- Given a chain of n matrixes (A1, A2 .... An) where matrix Ai is of Dimension $P_i - 1 * P_i$.

  The MCM problem is to fully parenthesize the chain such that the total number of scalar multiplication are minimized.



Topic : Dynamic Programming

#Q. How many ways to multiply chain n matrices?

K = 1 K=2

↓ ↓

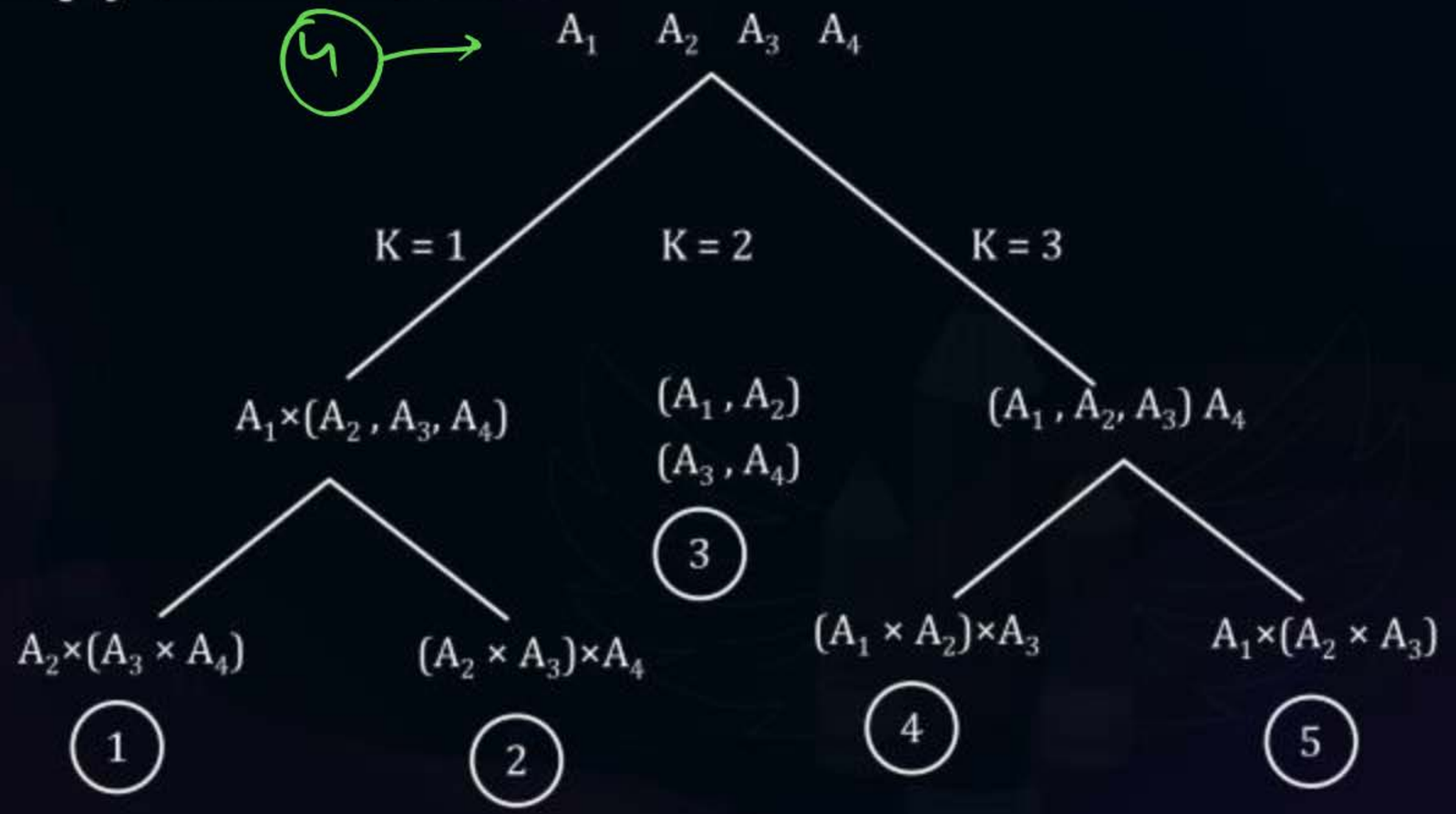
ABC

(1) A*(B*C)

(2) ~~(B*C)*C~~

(A*B)*C

3



5 ways

Topic : Dynamic Programming

Generalized Results:-

Total no. of ways to multiply a chain of (n + 1) matrix ⇒ Catalan number (5)

E.g. N = 3

↓

4 matrix

$$= \frac{1}{4} \times {}^{6}C_3$$

$$= \frac{1}{4} \times \frac{6 \times 5 \times 4 \times 3}{3! \times 3!}$$

$$= \frac{6 \times 5}{5} = 5$$

$$\boxed{\frac{1}{(n+1)} \times {}^{2n}C_n}$$

n +1

n ⇒ n-1

Topic : Dynamic Programming

n Matrix $\rightarrow \frac{1}{n} \times^{2(n-1)} C_{(n-1)}$

Topic : Dynamic Programming

Derivation

~~Deamination~~ of DP based Recurrence for men:

Let the resultant matrix Ai j be of product

$(A_i * Ai+1 * A_{i+2} \ldots\ldots A_j)$

Chain → Result Ai j

$A_1 \times A_2 \times A_3 \ldots\ldots A_n \Rightarrow A_{1\times n}$

$k = (1 \text{ to } n-1)$

Topic : Dynamic Programming

Any optimal ~~Parameterization~~ Parenthesization must split the chain about the matrix A_k & A_{k+1} ~~rule~~ such that the number of scaler multiplications are minimum.

Topic : Dynamic Programming

DP Recurrence

Let m[i, j] represent the number of scalar multiplication to get the resultant matrix Ai j.

$[A_i \times A_{i+1} \ldots\ldots\ldots A_k * \ldots\ldots A_j] \Rightarrow A\ i\ j \rightarrow M\ [i, j]$

Split at k

$(A_i \times A_i + \ldots\ldots A_k) \times (A_{k+1} \ldots\ldots\ldots A_j)$ Where $\rightarrow$ $i \leq k \leq (j-1)$

$m\ [i\ ,\ k]$ $m\ [k+1, j]$

Topic : Dynamic Programming

$$M[i, j] = \min \{m[i,k] + m[k+1, j] + P_{i-1} * P_k * P_j\}$$

$i \le k \le j-1$

$(A_i \ldots\ldots\ldots\ldots A_k) * (A_{k+1} \ldots\ldots\ldots\ldots A_j)$

$P_{i-1} * P_i$ $\quad$ $P_{k-1} * P_k$

$P_{i-1} * P_k$

$P_k * P_{k+1}$ $\quad$ $P_{j-1} * P_j$

$P_k * P_j$

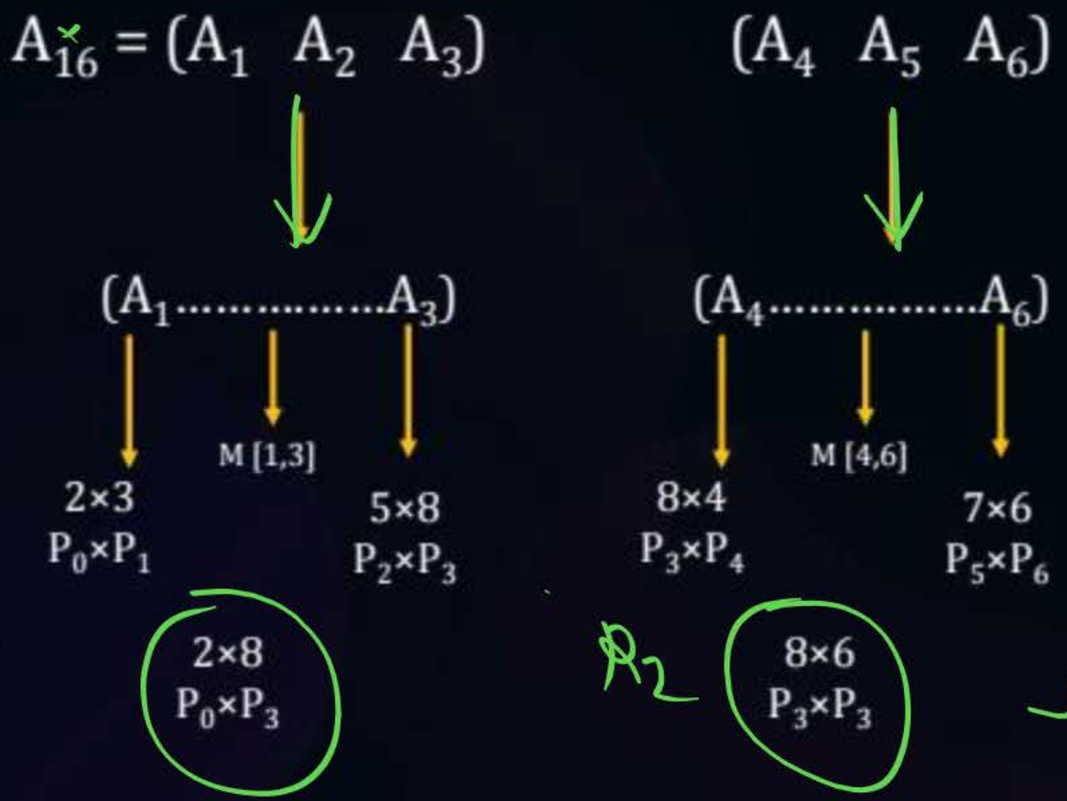
$R_1 * R_2$

$A_1 A_2 A_3 A_4$

Topic : Dynamic Programming

Example:

M[1,6]



Given,

$A_1 \rightarrow P_0 \times P_1 \rightarrow 2 \times 3$
$A_2 \rightarrow P_1 \times P_2 \rightarrow 3 \times 5$
$A_3 \rightarrow P_2 \times P_3 \rightarrow 5 \times 8$
$A_4 \rightarrow P_3 \times P_4 \rightarrow 8 \times 4$
$A_5 \rightarrow P_4 \times P_5 \rightarrow 4 \times 7$
$A_6 \rightarrow P_5 \times P_6 \rightarrow 7 \times 6$

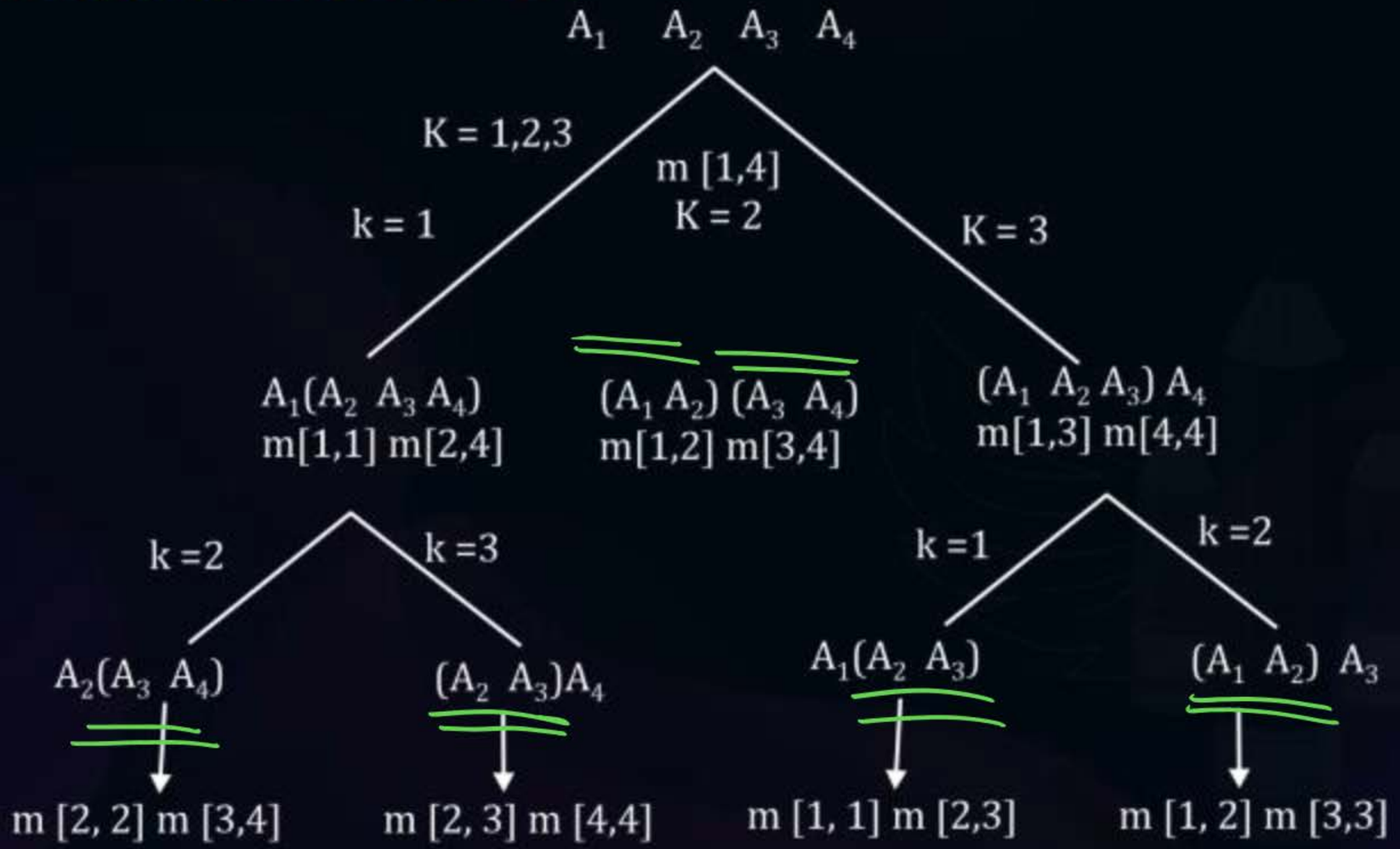
: $[P_0 \ldots\ldots\ldots P_n]$

Number of both Resultants = $P_0 * P_3 \times P_6$
$= 2 \times 8 \times 6$

i =1, k = 3, j = 6

Topic : Dynamic Programming

Are there overlapping Subproblems?



Topic : Dynamic Programming

35.7%

↓

174

Top Down: Recursion Approach

E.g. $A_1 A_2 A_3 A_4 = [2\times3,\ 3\times5,\ 5\times8,\ 8\times4]$

P_0 P_1 P_2 P_3 P_4

P : [2, 3, 5, 8, 4]

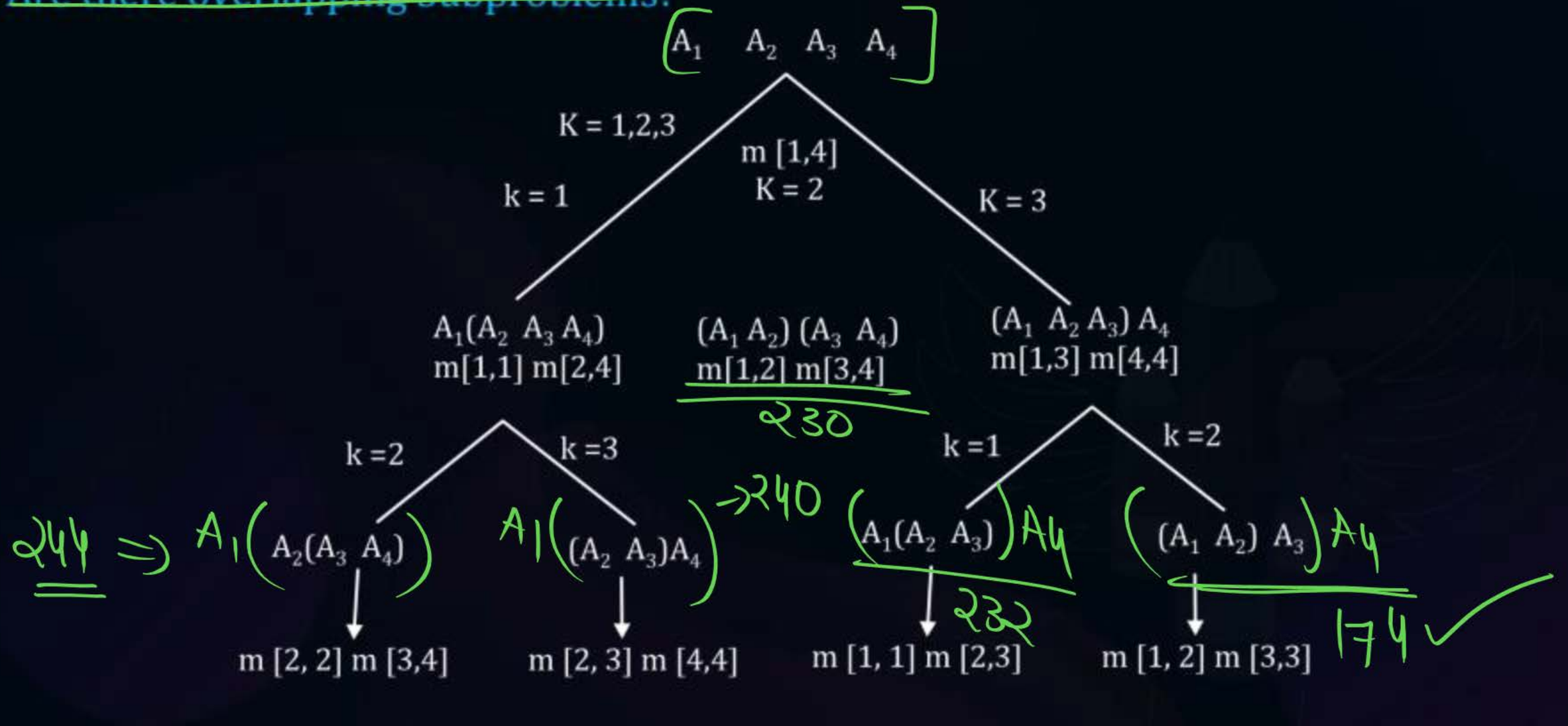
min

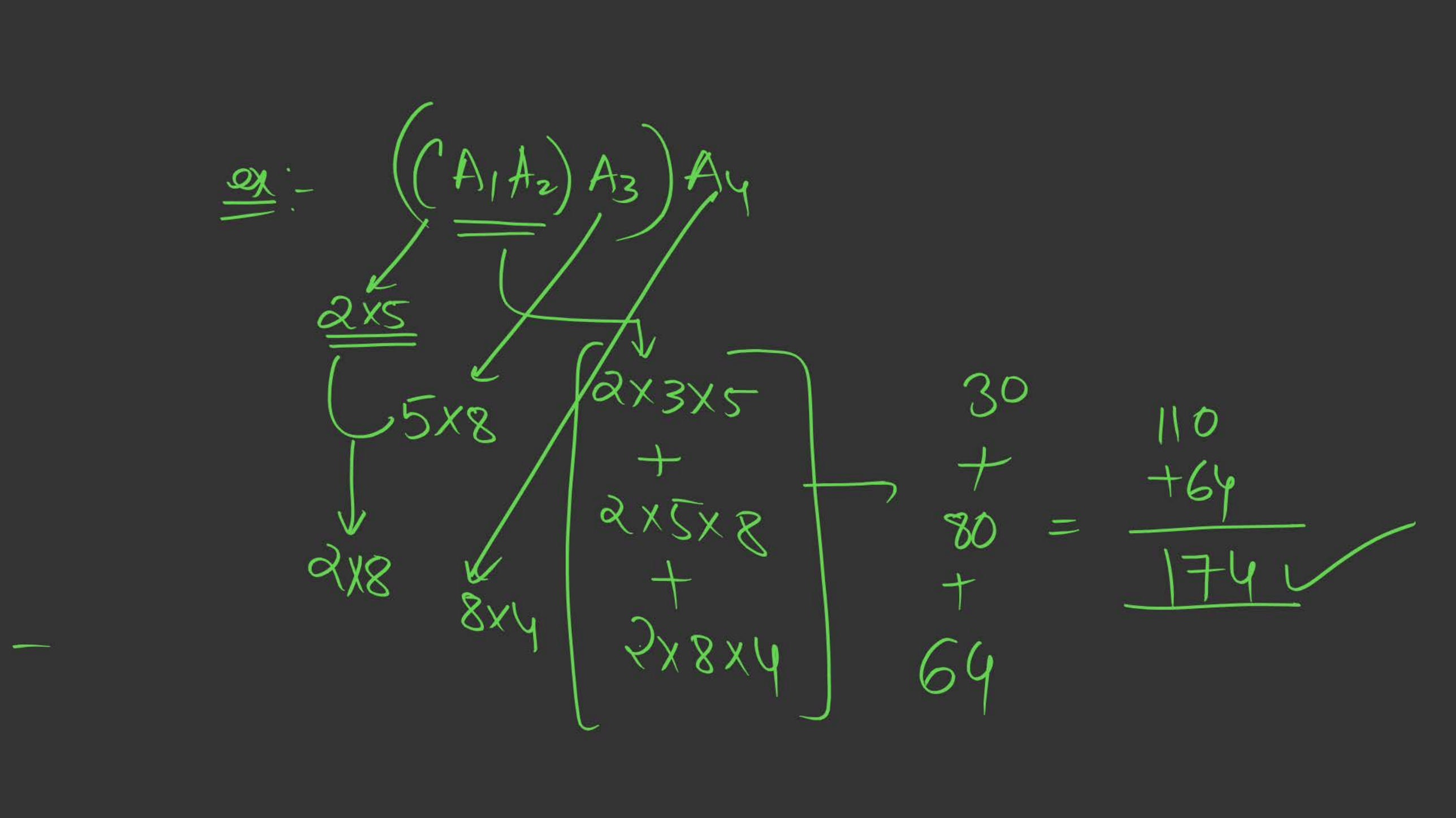
Topic : Dynamic Programming

#Q. The minimum no. of scaler multifaction required to ~~mull~~ multiply The chain $A_1A_2A_3A_4$ = ?

Topic : Dynamic Programming

Are there overlapping Subproblems?




ex :-
((A1 A2) A3) A4
2x5
5x8
2x8
8x4
2x3x5
+
2x5x8
+
2x8x4
30
+
80
+
64
=
110
+64
174

Topic : Dynamic Programming :(DP)

MCM Recurrence

$M[i, j] \rightarrow \quad A_i \times A_{i+1} \ldots\ldots\ldots\ldots Aj$

$P_{i-1} \times P_i \quad P_i \times P_{i+1} \ldots. \quad P_{i-1} \times P_j$

$P \rightarrow$ dimensions

Topic : Dynamic Programming :(DP)

K→ Breakup point

$$(A_i \times A_{i+1} \ldots\ldots\ldots\ldots A_k) * (A_{k+1} \ldots\ldots\ldots\ldots\ldots\ldots A_j) = m[I, j]$$

$P_{i-1} \times P_k$

$M [i,k]$

$M [k+1, j]$

$P_k * P_j$

$$i \leq k \leq j - 1$$

$$M [i,j] = \text{main} \{M [I, j] + m [k+i, j] + P_{i-1} * P_k * P_j \}$$

Topic : Dynamic Programming :(DP)

Algorithm Matrix- Chain Product(p)

```
1.    n ← length [p] –1
2.    for i ← 1 to n
3.          do m [i ,i] ← 0
4.    for l ← 2 to n
5.          for I ← 1 to n – l + 1
6.                J ← i + l – 2
7.          M [i ,j] ← ∞
```

l → chain length

Topic : Dynamic Programming :(DP)

Algorithm Matrix- Chain Product(p)

```
8.        for k ← i  to j –1
9.            q ← m [i, k] + m [k+1, j] + Pi-1 Pk Pj
10.               If (q < m [i, j])
11.   then m[i,j] ← q
12.   s [i , j] ← k
13.   return m and s
```

$i \le k \le j-1$

Topic : Dynamic Programming :(DP)

Complexity Analysis of Top– down MCM (DP approach)

Assume n → no. of matrix

1. Time Complexity: $O(n^3)$ → for every length chain and for every break-up point in it.

2. Space complexity:- $O(n^2)$

Brute force / Enumeration: TC : $\Omega(2^n)$

Topic : Dynamic Programming :(DP)

6. Sum of subset: (SOS)

Problem Statement:- Given a set of n-elements (integers) and also another elements (integer) 'M'.

The sum of subsets (~~SDS~~ SOS) problem is to determine If there ~~exits~~ exists a subset of the given elements whose sum equals to 'M'.

Topic : Dynamic Programming :(DP)

SOS→ Decision Problem

Optimization problems (min/max)

1. Bellman – Ford→ Minimize path cost (SSSP)
2. Floyd Warshall → Minimize path cost (APSP)
3. 0/1 knapsack → Maximize Profit
4. Longest common Subsequence (LCS) → Maximize length of common subsequence.
5. Matrix chain multiplication (MCM) → Minimize scalar multiplication

Topic : Dynamic Programming :(DP)

SOS→ Very similar to 0/I knapsack problem:

Eg.1. N = 5

A =[50, 30, 10, 40, 20]

M = 50

Subsets	Index
{50}	{1}
{30, 20}	{2, 5}
{10, 40}	{3, 4}

Topic : Dynamic Programming :(DP)

Enumeration

Brute Force:-

Check every subset→ ~~2n~~ 2^n → subset

TC: $O(2^n)$

Topic : Dynamic Programming :(DP)

DP: SOS→ Top-Down (Recurrence of SOS DP)

Target Sum

n, A[Ai..... An], M, → X[1.....n]

Let SOS (n, m) represent the solution to the SOS, with n elements and target sum (M).

SOS (n, M) = True/ False = 0/1

Topic : Dynamic Programming :(DP)

SOS (n, m) = T {There exits a subset from n elements that sum to M}

= F { There does not exits}

OR

F or F → F

F or T → T

T or F → T

T or T → T

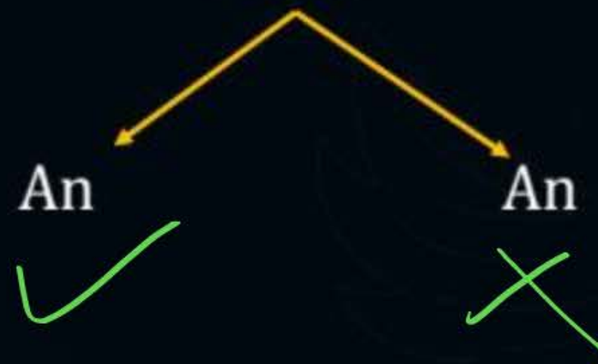
Topic : Dynamic Programming :(DP)

A = [1,1,1,1,1]
M = 3

SOS (n, M) = SOS (n–1, M) , An > M {skip/ reject An→ X}

{ SOS (n–1, M)
OR
SOS (n –1, M –An) }

An ≤ M
Choice



10
1
3
7

If we can get to sum M from either taking An or Rejecting An

Topic : Dynamic Programming :(DP)

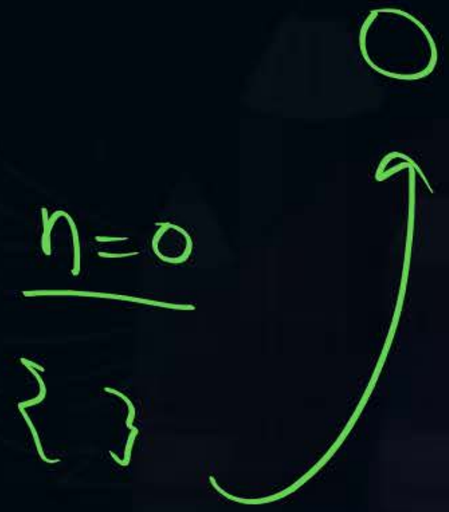
Boundary case

or

Base condition

SOS (n, m) = False , n = 0 , M > 0

SOS (n, m) = True , M = 0 , n ≥ 0



Topic : Dynamic Programming :(DP)

Explanation

SOS (n, m) = True ?

M = 0, n = 5

Take decision of each of n elements
(not mandatory to take or leave them)

SOS

n = 0, m = 0

{True}

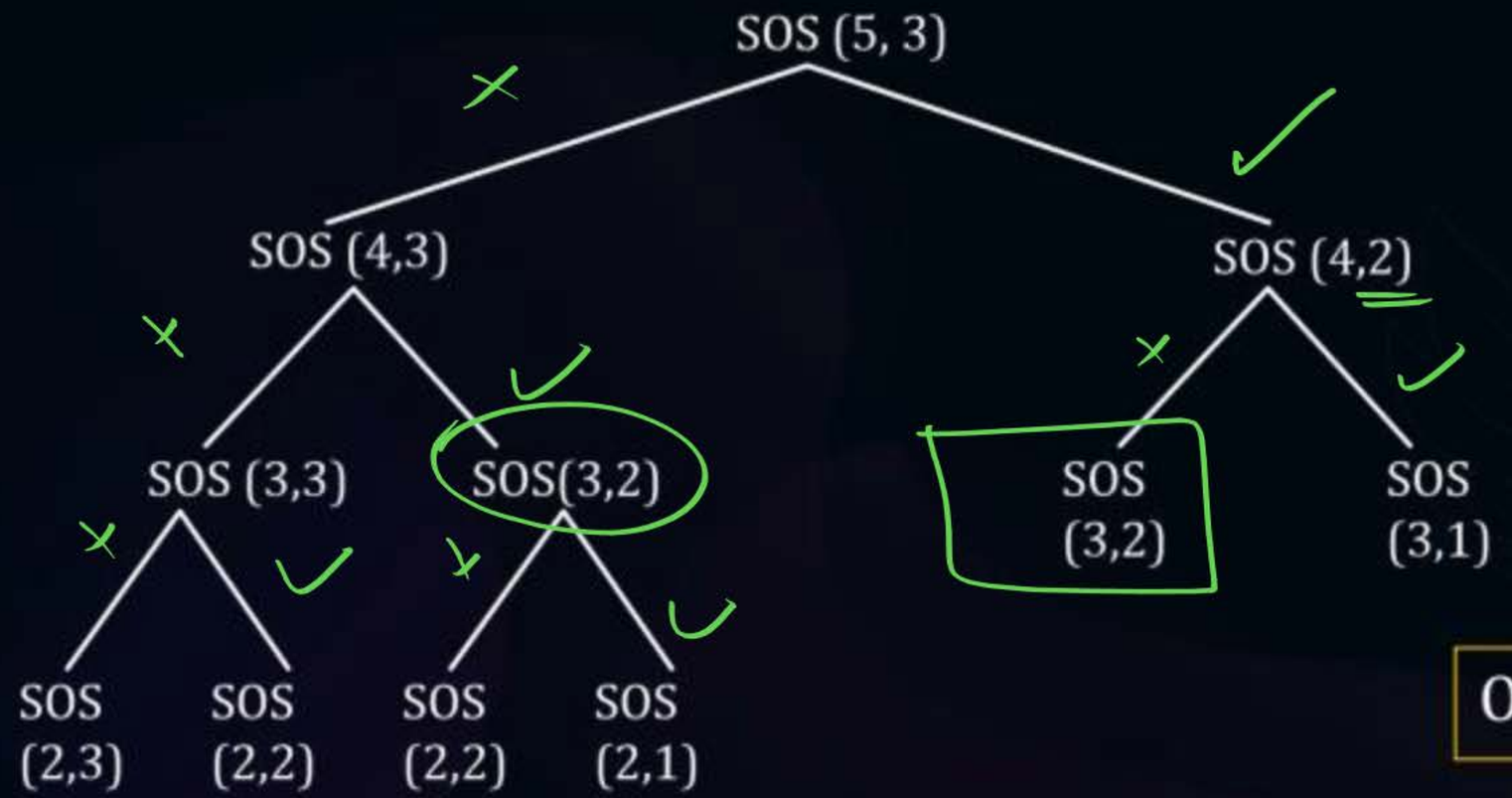
-

m=0

Topic : Dynamic Programming :(DP)

TOP – Down

E.g.:- n =5, A =[1, 1, 1, 1, 1] M = 3



Overlapping subproblems

Topic : Dynamic Programming :(DP)

SOS → Bottom Up (Tabulation Method)

Given:- n, m x [0, ……n 0…..m]

X[I,j] = True /False

A particular I,j call form x[,]

Whether false exits a subset from the 'i' elements that sum to 'j'.

E.g.2. A = [2, 8, 4, 11, 9]

n= 5,

m = 6

Topic : Dynamic Programming :(DP)

M

Ai	X	0	1	2	3	4	5	6
2	0	T	F	F	F	F	F	F
n 8	1	T	F	T	F	F	F	F
	2	T	F	T	F	F	F	F
4 →	3	T	F	T	F	T	F	T
11	4	T	F	T	F	T	T	T
9	5	T	F	T	F	T	T	T

$(n+1) \times (m+1)$

Final ans

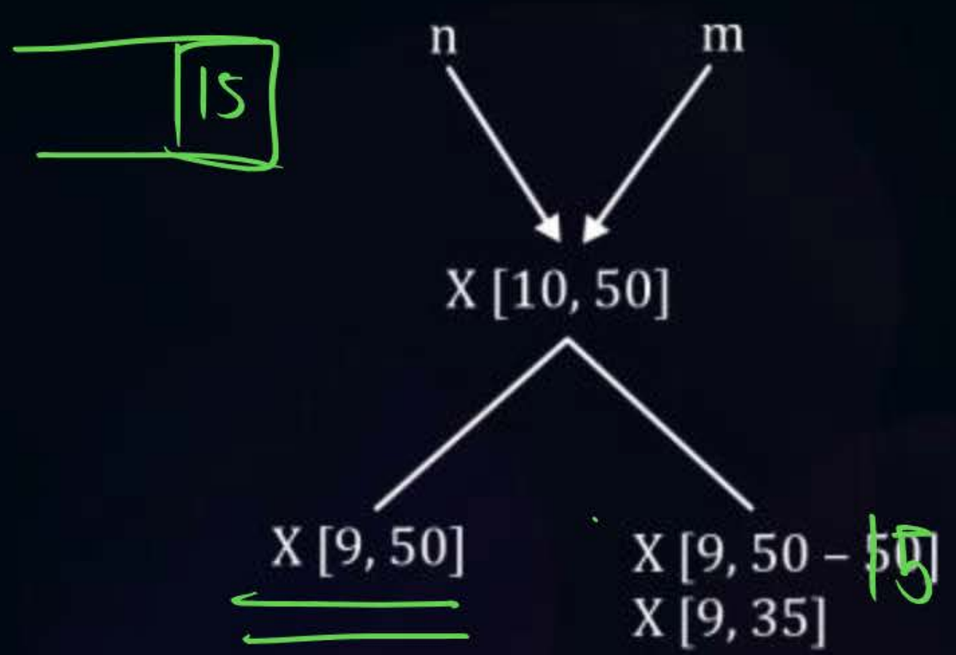
Topic : Dynamic Programming :(DP)

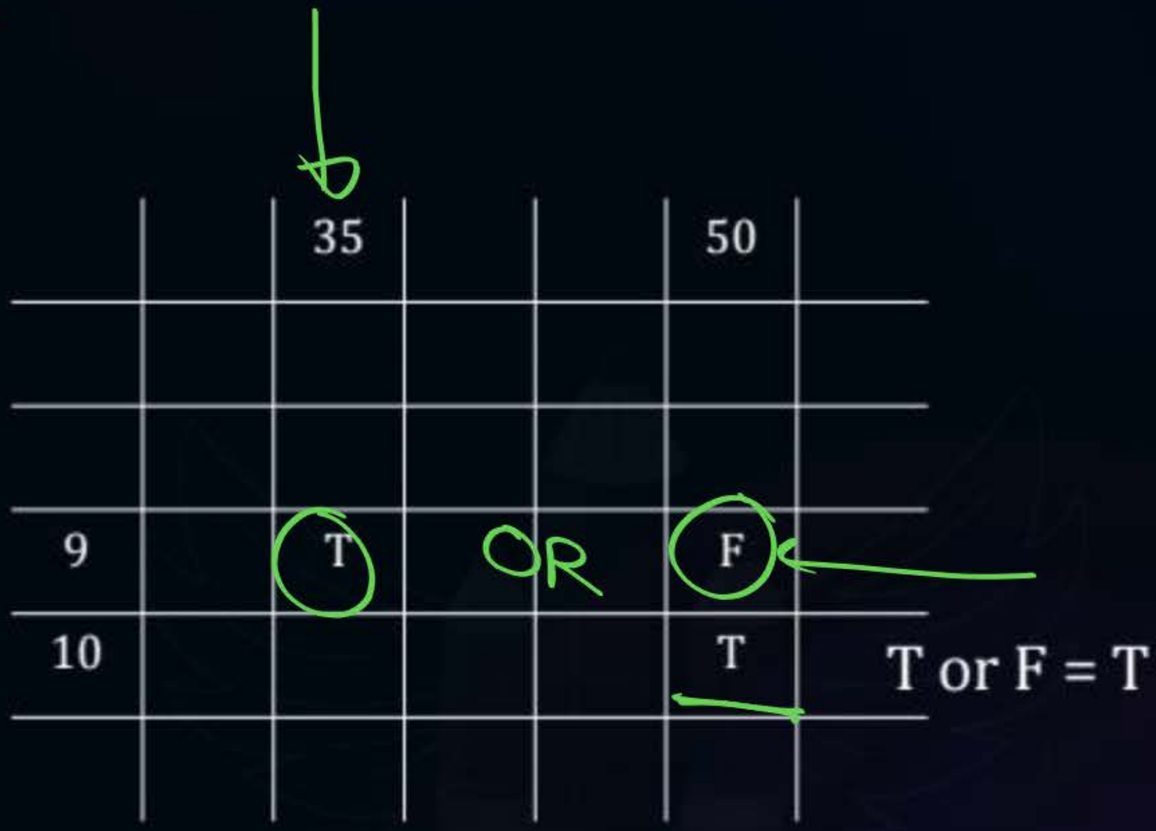
Imp Shortest

If A [i, j] = True then the entire column 'j' can be set to True.

Topic : Dynamic Programming :(DP)

For large example, how to use recoherence. recurrence





Topic : Dynamic Programming :(DP)

DP: SOS: Tabulation (Bottom -up) Code

```
Algo SOS (n, M, A)          A [1.....n]
{
    X [0....n,   0.....M] → Auxiliary Space
        for i = (0 to n):
    {
        for j = (0 to m)
        {
            if (i ≥ 0 and j =0)
            {
                X [i, j] = True
            }
                else  if [i = 0 and j > 0]
                }
                else if (A [i] > j)
                {
                    x [I,j] = x [i–1  , j]
                }
                // Choice  Ai ≤ j
            else
            {
                x [i,j] = (x[i–1, j] or X[i–1, j –A [i]])
            }
        }
    }
}
```

95% same as 0/1 Knap

i = 0 and j > 0] → X[i, j] = False

Topic : Dynamic Programming :(DP)

Complexity Analysis of Bottom- up (DP) Approach of SOS:

1. Time complexity : O(n*m)
2. Space complexity : O(n*m)

Note:

[redacted]

2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

MCM ✓

SOS ✓

THANK - YOU